Deploy aplikacji webowej

1. Cel zadania:
   Zadanie ma na celu postawienie lokalnie aplikacji webowej na serwerze przy pomocy Dockera.
2. Sposób rozwiązania:
   -Utworzenie repozytorium na Githubie
   -Utworzenie Dockerfile do stworzenia obrazu
   -Przygotowanie środowiska na Proxmoxie
   -Test obrazu z Dockerfile
   -Optymalizacja obrazu
   -Utworzenie Docker Compose (compose.yaml)
   -Uruchomienie usługi na maszynie wirtualnej
   -Testowanie wdrożenia

2.2. Utworzenie Dockerfile

Jeszcze nigdy tego nie robiłem więc zrobiłem rekonesans w dokumentacji Dockera.

Etap budowania:
 -Ustawienie obrazu bazowego Node.js 20 LTS w wersji slim która jest oparta na Debianie co zapewnia lepszą kompatybilność oraz stabilność:

```
#Set image node.js 20 Lts Slim
FROM node:20-slim AS builder
```

 -Instalacja Gita aby pobrać repozytorium (apt-get ze względu że operujemy na Debianie):

```
#Install Git
RUN apt-get update && apt-get install -y git
```

 -Określenie folderu roboczego aplikacji:

```
#Define working directory
WORKDIR /app
```

-Sklonowanie repozytorium:

```
#Clone repo
RUN git clone https://github.com/Solvro/web-solvro-docs.git .
```

-Zmodyfikowanie wartości allowScripts z “false” na “true” oraz instalacja wymaganych bibliotek i skompilowanie strony do statycznych plików:

```
#Install dependencies by using npm and change param of allowScripts
RUN npm pkg set 'allowScripts.sharp=true' \
    npm ci \
    npm run build
```

Etap uruchomieniowy:

Ustawienie obrazu Nginx (wersja stable zapewnia ostatnią stabliną gałąź):

```
#Set image of nginx
FROM nginx:stable AS serve
```

Kopiowanie dist z plikami statycznymi do katalogu Nginxa aby serwować stronę:

```
#Copy dist
COPY --from=builder /app/dist /usr/share/nginx/html
```

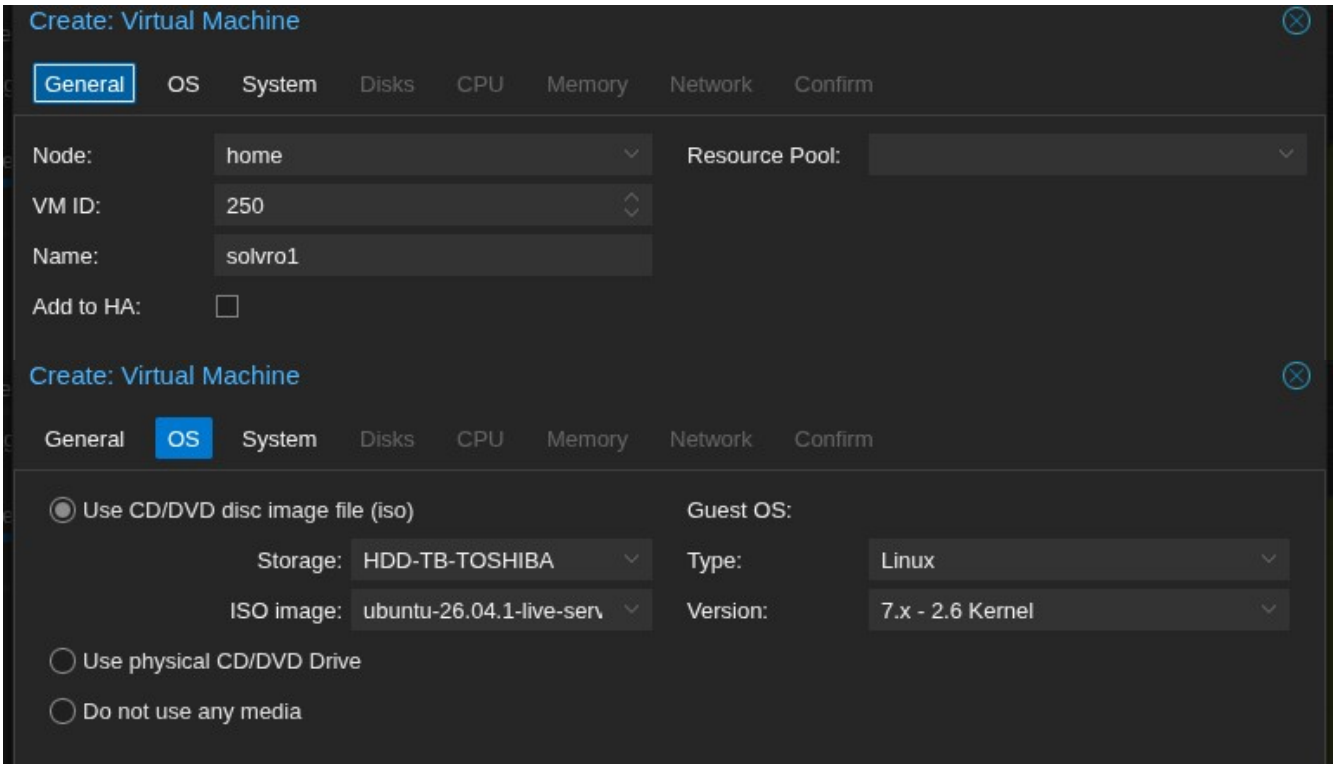
Ustawienie portu:

```
#Port to expose
EXPOSE 80
```

Nie wprowadzam CMD
aktualnie ponieważ zostawiam domyślną konfigurację

2.3 Przygotowanie środowiska Proxmox

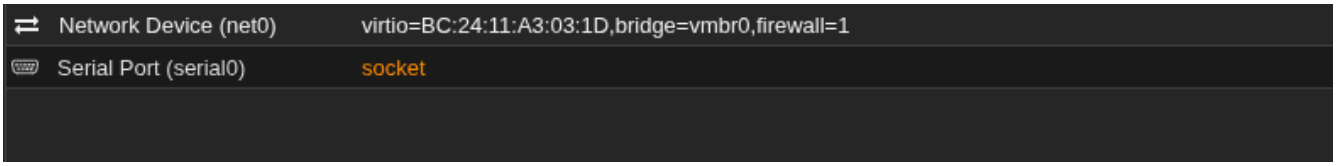
Zaczynamy od stworzenia wirtualnej maszyny z Ubuntu Server



Następnie dokonujemy instalacji Ubuntu Server na maszynie oraz aktualizujemy system.
Instalujemy Docker engine i compose.

Ale zanim tego dokonamy dodamy xterm.js jako konsole w proxmoxie by móc skopiować zalecaną instalację Dockera oraz link do githuba

W zakładce hardware dodajemy serial port na socket:



Następnie podpinamy serial port w grubie:

```
# For full documentation of the options in these files, see:
#   info -f grub -n 'Simple configuration'

GRUB_DEFAULT=0
GRUB_TIMEOUT_STYLE=hidden
GRUB_TIMEOUT=0
GRUB_DISTRIBUTOR=`( . /etc/os-release && echo ${NAME} )`
GRUB_CMDLINE_LINUX_DEFAULT=""
GRUB_CMDLINE_LINUX="console=tty0 consolettyS0, 115200"

# If your computer has multiple operating systems installed, then you
# probably want to run os-prober. However, if your computer is a host
# for guest OSes installed via LVM or raw disk devices, running
# os-prober can cause damage to those guest OSes as it mounts
# filesystems to look for things.
#GRUB_DISABLE_OS_PROBER=false

# Uncomment to enable BadRAM filtering, modify to suit your needs
# This works with Linux (no patch required) and with any kernel that obtains
# the memory map information from GRUB (GNU Mach, kernel of FreeBSD ...)
#GRUB_BADRAM="0x01234567,0xfefefefe,0x89abcdef,0xefefefef"

# Uncomment to disable graphical terminal
#GRUB_TERMINAL=console
```

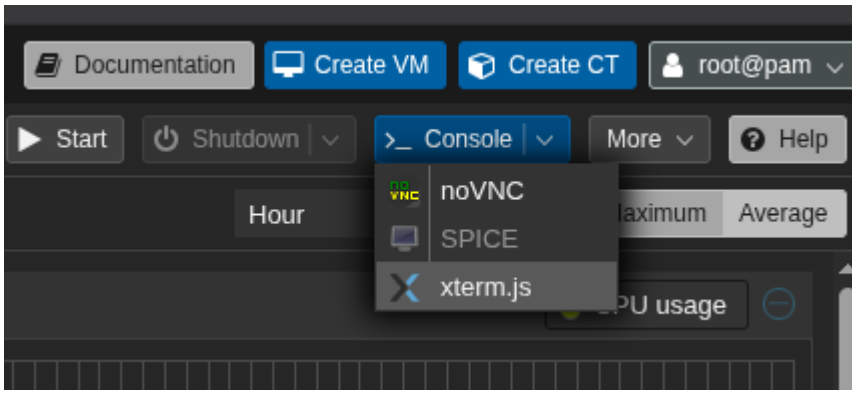
Aktualizujemy grub:

```
admin1@ubuntudocker:~$ sudo update-grub
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/kdump-tools.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-7.0.0-38-generic
Found initrd image: /boot/initrd.img-7.0.0-38-generic
Found linux image: /boot/vmlinuz-7.0.0-30-generic
Found initrd image: /boot/initrd.img-7.0.0-30-generic
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
admin1@ubuntudocker:~$
```

Włączamy serwis do nasłuchu na serial porcie:

```
admin1@ubuntudocker:~$ sudo systemctl enable serial-getty@ttyS0.service
Created symlink '/etc/systemd/system/getty.target.wants/serial-getty@ttyS0.service' → '/usr/lib/systemd/system/serial-getty@.service'.
admin1@ubuntudocker:~$ sudo systemctl start serial-getty@ttyS0.service
admin1@ubuntudocker:~$
```

Mamy teraz konsole Xterm.js która umożliwia nam kopiowanie i wklejanie:



Teraz instalujemy Dockera poprzez dodanie oficjalnego klucza GPG i repozytorium:

```
admin1@ubuntudocker:~$ sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
[sudo: authenticate] Password:
```

Instalujemy pluginy Dockera:

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin   ocker-compo
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30                  linux-modules-7.0.0-30-generic
  linux-headers-7.0.0-30-generic          linux-tools-7.0.0-30
  linux-image-unsigned-7.0.0-30-generic   linux-tools-7.0.0-30-generic
  linux-main-modules-zfs-7.0.0-30-generic
Use 'sudo apt autoremove' to remove them.

Installing:
  containerd.io         docker-ce      docker-compose-plugin
  docker-buildx-plugin  docker-ce-cli

Installing dependencies:
  docker-ce-rootless-extras  pigz

Suggested packages:
  cgroupfs-mount  | cgroup-lite  docker-model-plugin
```

Weryfikujemy instalacje:

```
admin1@ubuntudocker:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:11a3eb1597e864c51b3f56733cb2e8a90bf0c284bfc05f9444d81c558bc848aa
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

admin1@ubuntudocker:~$ docker compose version
Docker Compose version v5.6.1
```

Instalujemy jeszcze gita aby pobrać repozytorium z Dockerfilem:

```
admin1@ubuntudocker:~$ sudo apt install git-all
```

I dodajemy nasze repozytorium:

```
admin1@ubuntudocker:~/Docker-project$
admin1@ubuntudocker:~/Docker-project$
admin1@ubuntudocker:~/Docker-project$
ps://github.com/Bartek21371/Solvro-recruitmente https://github.com/Bartek21371/Solvro-recruitment
Cloning into 'Solvro-recruitment'...
remote: Enumerating objects: 462, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 462 (delta 1), reused 12 (delta 1), pack-reused 450 (from 2)
Receiving objects: 100% (462/462), 51.13 MiB | 16.49 MiB/s, done.
Resolving deltas: 100% (16/16), done.
admin1@ubuntudocker:~/Docker-project$
```

2.4 Testujemy obraz z Dockerfile

Przechodzimy do katalogu w którym znajduje się Dockerfile i wykonujemy budowanie obrazu:

```
udmin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$ nano Dockerfile
 docker build --no-cache -t solvro-docs .
[+] Building 565.3s (9/11)                                          docker:default
[+] Building 565.9s (9/11)                                          docker:default
[+] Building 705.0s (12/12) FINISHED                                docker:default
 => [internal] load build definition from Dockerfile                         0.1s
 => => transferring dockerfile: 577B                                         0.0s
 => [internal] load metadata for docker.io/library/nginx:stable              2.5s
 => [internal] load metadata for docker.io/library/node:20-slim              2.6s
 => [internal] load .dockerignore                                            0.3s
 => => transferring context: 2B                                              0.0s
```

W trakcie należało poprawic ten fragment na ten:

```
#Install dependencies by using npm and change param of al#Install dependencies by using npm and change param of allowScripts
RUN npm pkg set 'allowScripts.sharp=true' \             RUN npm pkg set 'allowScripts.sharp=true' && \
    npm ci \                                                npm ci && \
    npm run build                                           npm run build
```

Ponieważ instrukcje muszą wykonywać się pokolei gdy poprzednia zakończyła się sukcesem.

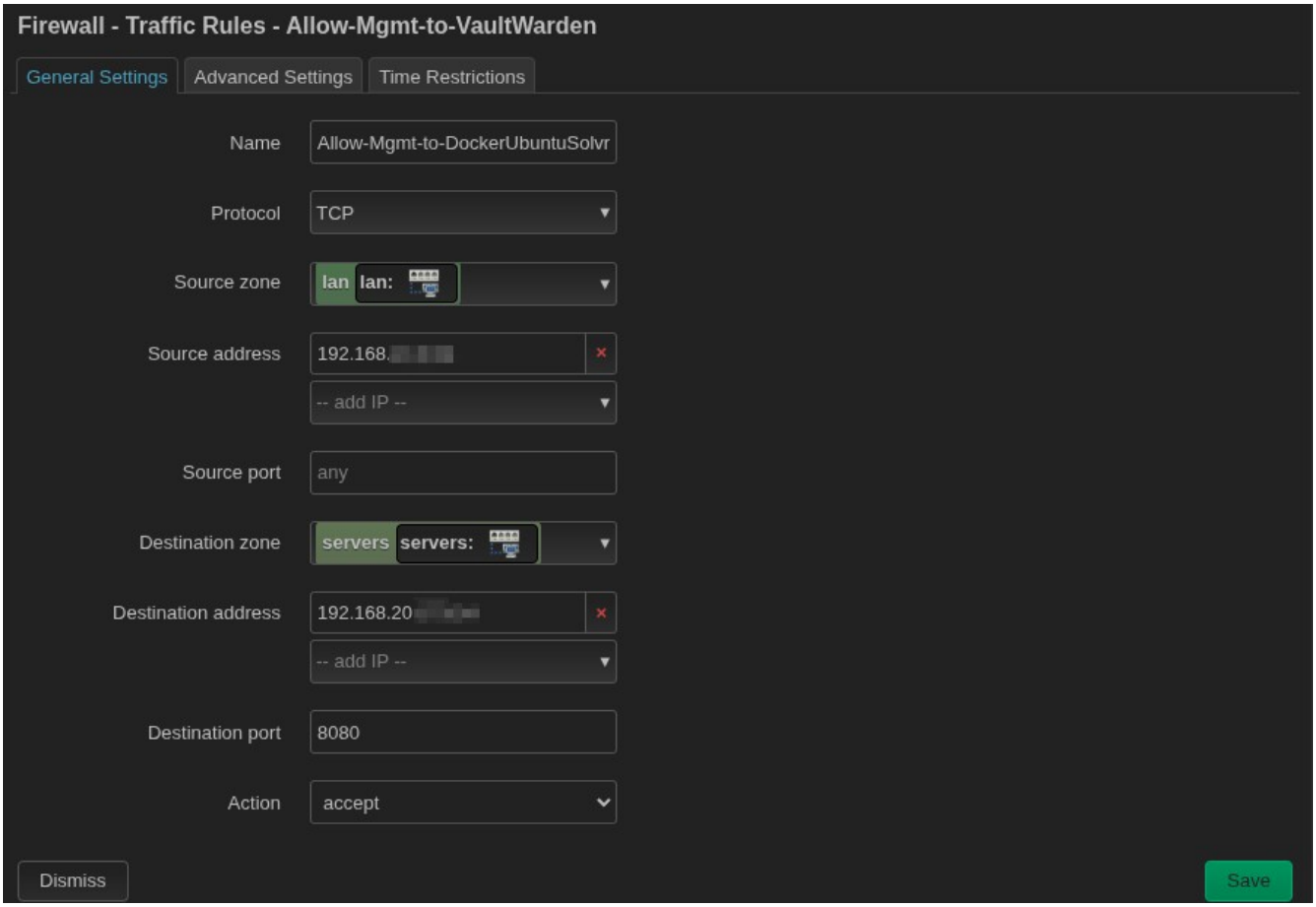
Nasz obraz powstał:

```
[sudo: authenticate] Password:
                                                          i Info →  U In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   11a3eb1597e8       25.9kB         9.49kB    U
solvro-docs:latest   5b83748d15d3        306MB          88.4MB
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$
```

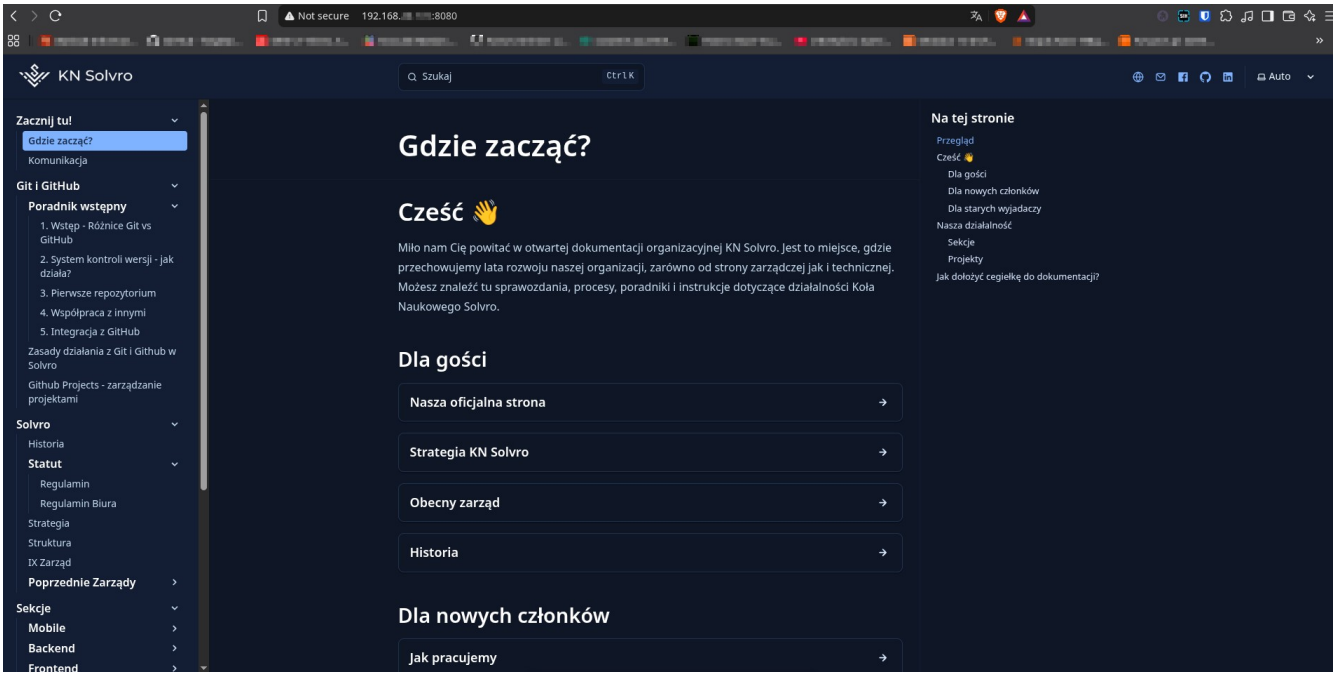
Uruchamiamy teraz kontener:

```
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$ sudo docker run -d --name solvro-docs -p 8080:80 solvro
-docs
```

Teraz muszę dodać w firewallu zasadę która da mi dostęp do usługi ponieważ posiadam Proxmoxa w osobnym izolowanym vlanie:



Nasza strona działa:



Sprawdzimy teraz działanie kontenera:

```
er ps1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$ dock
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
 docker psntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$ sudo
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS         PORTS                                       NAMES
0d41764434c3   solvro-docs   "/docker-entrypoint.…"   7 minutes ago   Up 7 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   solvro-doc
s
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$
```

```
recruitment/deploy_aplikacji_webowej$ docker logs solvro-docs
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$
 docker logs solvro-docs     project/Solvro-recruitment/deploy_aplikacji_webowej$ sudo
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/21 22:23:05 [notice] 1#1: using the "epoll" event method
2026/10/21 22:23:05 [notice] 1#1: nginx/1.30.6
2026/10/21 22:23:05 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/10/21 22:23:05 [notice] 1#1: OS: Linux 7.0.0-38-generic
2026/10/21 22:23:05 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/10/21 22:23:05 [notice] 1#1: start worker processes
2026/10/21 22:23:05 [notice] 1#1: start worker process 30
192.168.[redacted] - - [21/Oct/2026:22:28:07 +0000] "GET / HTTP/1.1" 200 82900 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
192.168.[redacted] - - [21/Oct/2026:22:28:07 +0000] "GET /_astro/index.UgfoG2xB.css HTTP/1.1" 200 70641 "http://192.168.[redacted]:8080/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
192.168.[redacted] - - [21/Oct/2026:22:28:07 +0000] "GET /_astro/hoisted.BH4GwA6V.js HTTP/1.1" 200 6404 "http://192.168.[redacted]:8080/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
192.168.[redacted] - - [21/Oct/2026:22:28:07 +0000] "GET /_astro/page.7qqag-5g.js HTTP/1.1" 200 2165 "http://192.168.[redacted]:8080/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
192.168.[redacted] - - [21/Oct/2026:22:28:07 +0000] "GET /_astro/Tabs.astro_astro_type_script_index_0_lang.3nBd5krW.js HTTP/1.1" 200 1606 "http://192.168.[redacted]:8080/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
192.168.[redacted] - - [21/Oct/2026:22:28:07 +0000] "GET /_astro/logo_white.CPzMjvTz.png HTTP/1.1" 200 28229 "http://192.168.[redacted]:8080/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
192.168.[redacted] - - [21/Oct/2026:22:28:07 +0000] "GET /_astro/logo_dark.CVKYDfDL.png HTTP/1.1" 200 52908 "http://192.168.[redacted]:8080/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
```

Jak widać kontener działa oraz w logach widoczne jest połączenie z mojego komputera do strony.

2.5 Optymalizacja obrazu

Zmienimy wersje nginx ze stable na stable-alpine ze względu na wielkość obrazu:

```
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:stable         a7675fcee220        242MB         66.5MB
nginx:stable-alpine  970364228b1e        102MB         29.4MB
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$
```

```
#Set image of nginx
FROM nginx:stable-alpine AS serve
```

Teraz porównajmy obrazy strony:

```
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
solvro-docs:latest   5b83748d15d3        306MB         88.4MB    U
 docker build --no-cache -t solvro-docs .
[+] Building 589.1s (10/11)                                      docker:default
```

nginx stable

```
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
solvro-docs:latest   5b34e0dd14ab        168MB         53.4MB
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$
```

nginx stable-alpine

Obraz z nginx-stable-alpine waży zdecydowanie mniej i to chcieliśmy osiągnąć.

2.6 Utworzenie Docker Compose

Teraz stworzymy plik compose.yaml aby uruchamiać usługę za pomocą docker compose up -d zamiast docker run.

```
 GNU nano 8.4
services:
  web:
    build: .
    ports:
      - "80:80"
```

Definujemy jakie usługi chcemy, w naszym przypadku to strona internetowa, build to “.” czyli compose.yaml jest w tym samym folderze co Dockerfile więc używamy bieżącego katalogu, port ustawiamy na “80:80”
dzięki temu wystarczy wpisać sam adres IP VM-ki.

Uruchamiamy teraz usługę przez compose więc zatrzymujemy kontener uruchomiony ręcznie oraz go usuwamy by uniknąć zajmowania portu:

```
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$ docker stop solvro-docs
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$ sudo docker rm solvro-docs
```

Sprawdzamy konfiguracje kompozycji:

```
er compose configer:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$ dock
name: deploy_aplikacji_webowej
services:
  web:
    networks:
      default: null
    build:
      context: /home/admin1/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej
      dockerfile: Dockerfile
    ports:
      - mode: ingress
        target: 80
        published: "80"
        protocol: tcp
networks:
  default:
    name: deploy_aplikacji_webowej_default
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$
```

Zmienimy w compose.yaml reguły restartu, aby kontener był automatycznie uruchamiany po restarcie VM:

```
 GNU nano 8.7.1                                co
services:
  web:
    build: .
    ports:
      - "80:80"
    restart: unless-stopped
```

2.7 Uruchomienie usługi na maszynie wirtualnej

Teraz zbudujemy obraz i uruchomimy stronę:

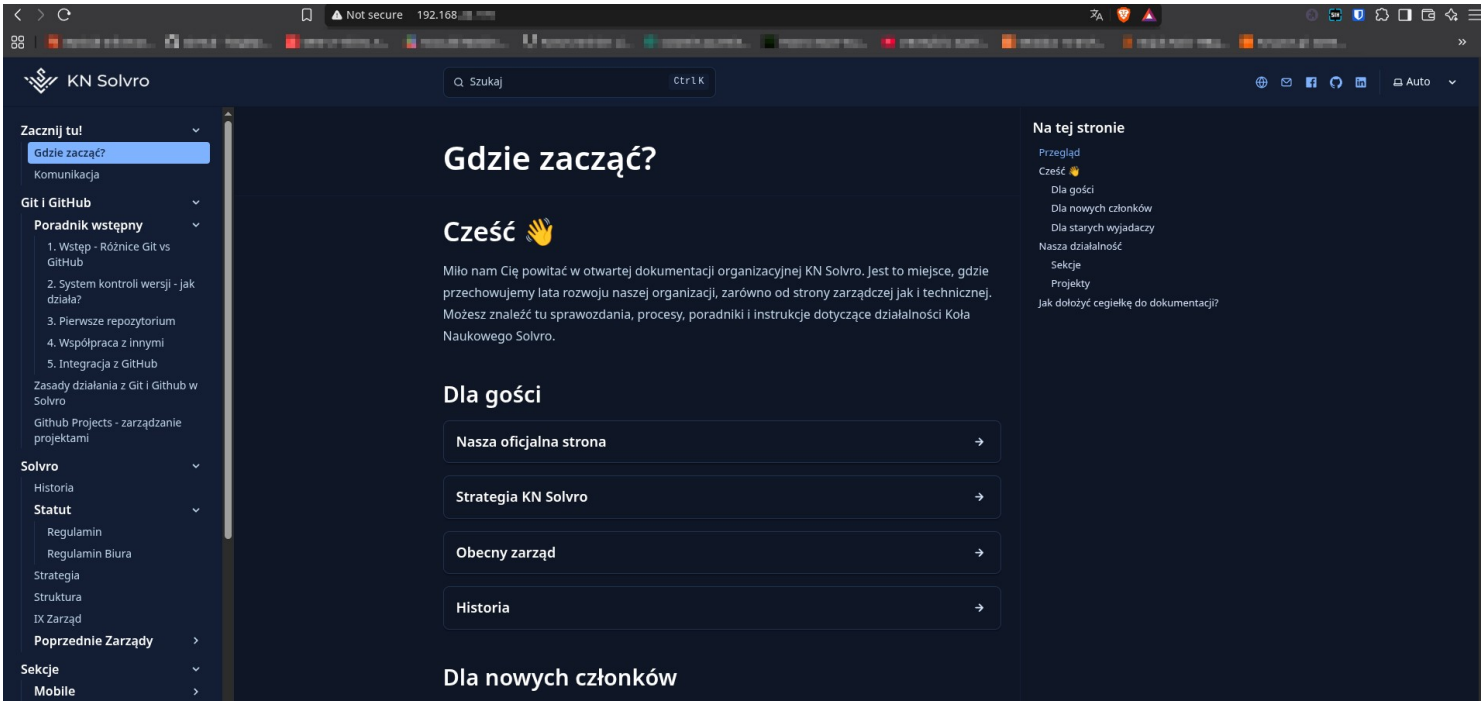
```
sdmin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$ sudo docker compose up -d --build
[+] Building 6.7s (13/13)
 => [internal] load metadata for docker.io/library/node:20-slim                1.4s
 => [internal] load .dockerignore                                              0.1s
 => => transferring context: 2B                                                0.0s
 => [builder 1/5] FROM docker.io/library/node:20-slim@sha256:2cf067cfed83d5ea95  0.5s
```

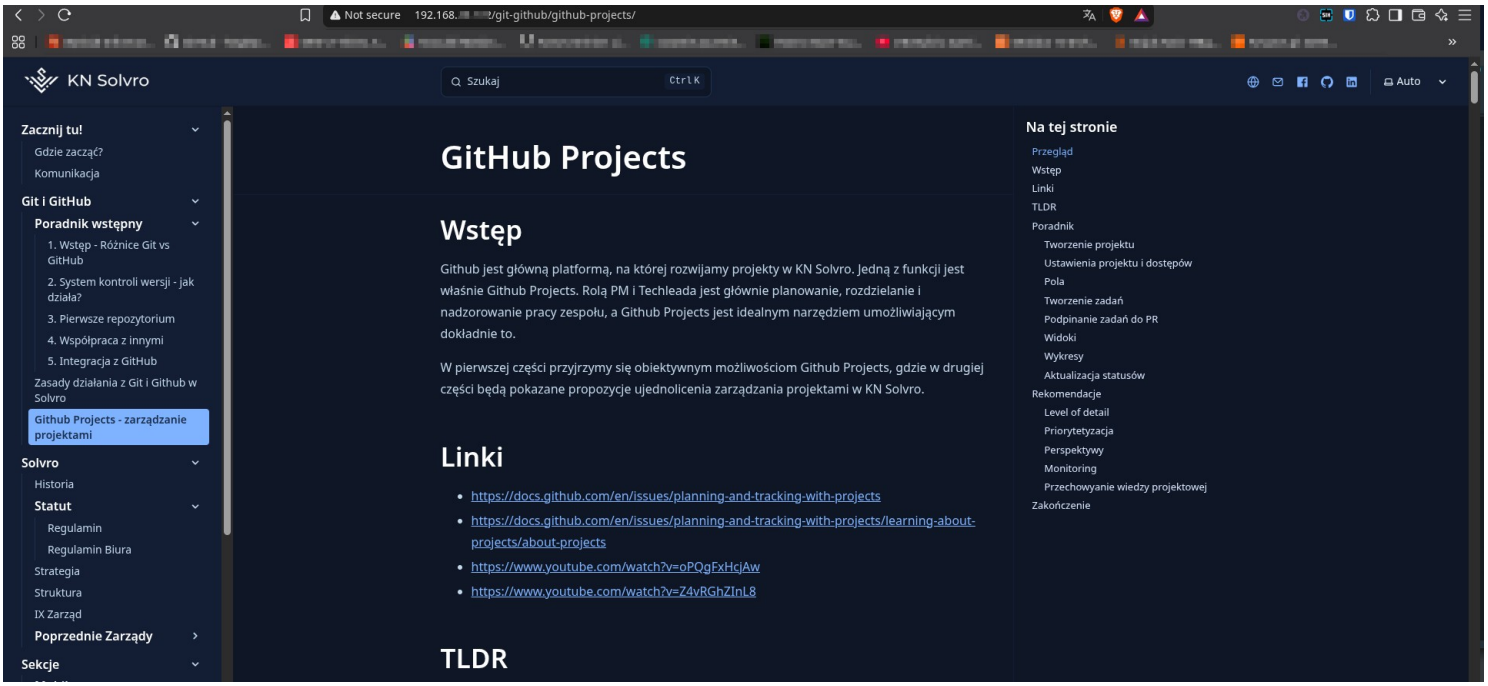
Sprawdzamy czy kontener działa:

```
 docker compose ps
NAME                        IMAGE                                                                  COMMAND                  SERVICE   CREATED         S
TATUS          PORTS
deploy_aplikacji_webowej-web-1   sha256:efc45b72d403a83d42653d5c7eea5b11d5c651a7ad1f9f42c8660de7b9f7cada   "/docker-entrypoint.…"   web       2 minutes ago   U
p 10 seconds
admin1@ubuntudocker:~/Docker-project/Solvro-recruitment/deploy_aplikacji_webowej$
```

2.8 Testowanie wdrożenia

Kontener działa teraz przejdziemy do strony:





Strona działa i nie wymaga portu tylko sam adres IP
logi również rejestrują ruch

```
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:32:43 +0000] "GET /_astro/preload-helper.CLcXU_4U.js HTTP/1.1" 200 985 "http://192.168.[redacted]/_astro/hoisted.BH4GwA6V.js" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:32:43 +0000] "GET /favicon.ico HTTP/1.1" 200 15086 "http://192.168.[redacted]/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:32:43 +0000] "GET /_astro/ui-core.CGm0rk41.js HTTP/1.1" 200 65170 "http://192.168.[redacted]/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:33:25 +0000] "GET /solvro/structure HTTP/1.1" 301 169 "http://192.168.[redacted]/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:33:25 +0000] "GET /solvro/structure/ HTTP/1.1" 200 106677 "http://192.168.[redacted]/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:34:03 +0000] "GET /git-github/intro/1-intro/ HTTP/1.1" 200 74418 "http://192.168.[redacted]/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:34:03 +0000] "GET /_astro/solvro-git-logo.mVDZSxAb_ZIJC5.webp HTTP/1.1" 200 26688 "http://192.168.[redacted]/git-github/intro/1-intro/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:34:04 +0000] "GET /git-github/github-projects HTTP/1.1" 301 169 "http://192.168.[redacted]/git-github/intro/1-intro/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:34:04 +0000] "GET /git-github/github-projects/ HTTP/1.1" 200 100237 "http://192.168.[redacted]/git-github/intro/1-intro/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
web-1  | 192.168.[redacted] - - [21/Oct/2026:23:34:04 +0000] "GET /_astro/link-project.B8poBafK_Z7FvSH.webp HTTP/1.1" 200 26814 "http://192.168.[redacted]/git-github/github-projects/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36" "-"
```

3.Podsumowanie

Zadanie nauczyło mnie pracy z Dockerem, w tym tworzenia Dockerfile oraz Docker Compose. Aplikacja webowa została wdrożona na moim serwerze Proxmox, na maszynie wirtualnej z Ubuntu Server.
Podczas realizacji stworzyłem repozytorium do zadania, a następnie plik Dockerfile, na podstawie którego utworzyłem obraz kontenera.
Przygotowałem środowisko Proxmox dla mojej maszyny wirtualnej. Zainstalowałem Ubuntu Server i Dockera, skonfigurowałem konsolę xterm.js oraz zainstalowałem Gita, aby pobrać repozytorium z plikami aplikacji.
Stworzyłem obraz na podstawie Dockerfile i przetestowałem jego działanie. Obraz wymagał drobnej poprawki, którą udało mi się wprowadzić. Uruchomiłem kontener z aplikacją i dodałem regułę firewalla na routerze, aby umożliwić dostęp do aplikacji z mojej sieci. Sprawdziłem działanie strony, kontenera oraz logi.
Po uzyskaniu działającego obrazu przeszedłem do jego optymalizacji. Zamieniłem nginx:stable na nginx:stable-alpine, dzięki czemu udało się zmniejszyć rozmiar obrazu z 88,4 MB do 53,4 MB.
Następnie utworzyłem plik Docker Compose, który ułatwia uruchamianie aplikacji i zarządzanie nią. Po sprawdzeniu konfiguracji poprawiłem plik compose.yaml, dodając regułę restart, która pozwala na automatyczne ponowne uruchomienie kontenera.
Uruchomiłem aplikację za pomocą Docker Compose, wcześniej wyłączając działający kontener. Na koniec ponownie sprawdziłem działanie strony. Aplikacja działa poprawnie, a logi przekazują informacje o ruchu.

4. Środowisko wykorzystane do realizacji zadania

-Proxmox Virtual Environment 9.2.21
-Ubuntu Server 26.04.1
-Docker 29.9.2
-Node.js 20-slim
-Nginx 1.30.6 (stable-alpine)
-Aplikacja

5. Wykorzystane źródła:

https://docs.docker.com/get-started/docker-concepts/building-images/writing-a-dockerfile/

https://docs.docker.com/compose/

https://docs.docker.com/reference/dockerfile/#copy

https://docs.docker.com/desktop/setup/install/linux/

https://github.com/docker/compose

https://hub.docker.com/_/nginx

Autor: Bartosz Marcińczyk